

TECHNISCHER PROJEKTBERICHT · SPRINT 3

Agentic AI für SEO/GEO-Auditing

Sprint 3 — Fix & Deploy

Sprint 3 schließt den Loop: Der neue **Fix-Agent** macht aus den priorisierten Findings konkrete Patches, ein **Human-in-the-Loop-Gate** gibt sie frei, ein **sicherer Dry-Run-Deploy** wendet nur Freigegebenes an — ohne je eine Live-Seite zu berühren, jeder Lauf exakt reproduzierbar (Seed 42).

VERFASSER	Mohammad Alboush
MATRIKELNUMMER	201922227
STUDIENGANG	Master Informatik · Modulprojekt Agentic AI
TECHNOLOGIEN	Python 3.12 · CrewAI · Reasoning-Port (Mock/SAIA/Claude) · Pydantic v2 · SQLite · mypy --strict
STAND	Sprint 3 · Juni 2026 — abgeschlossen

Einordnung: Sprint 3 ergänzt die **Handlungs-Schicht** des Audit-Loops.

Sprint 1 misst, Sprint 2 lernt — Sprint 3 baut Baustein **05 Fix · HITL · Deploy**: aus dem Befund wird eine konkrete, freigegebene Maßnahme.

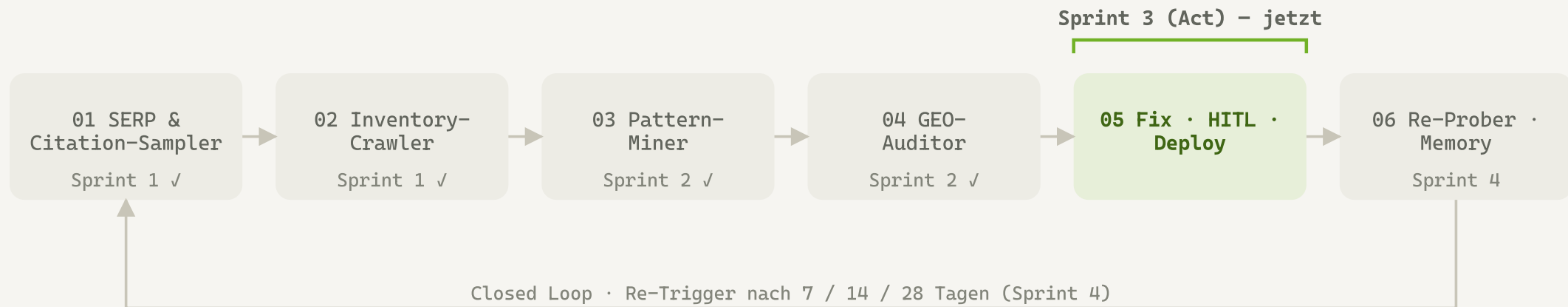


Abb. 1 – System-Kontext: sechs Bausteine. **Sprint 3 = Baustein 05** auf dem Fundament aus Sprint 1 + 2.

Ausgangslage: Der GEO-Auditor liefert priorisierte **AuditFinding**s — getestet und reproduzierbar. Sprint 3 setzt sie um: mit gleicher Architektur und Disziplin, aber neuer Sicherheits-Anforderung (kein Deploy ohne Freigabe).

Problemstellung: Der Auditor sagt, **was** zu tun ist — aber er **tut** es nicht.

Fragestellung Sprint 3: Lässt sich aus einem Befund eine *konkrete, anwendbare* Maßnahme ableiten und ausführen — *reproduzierbar* trotz Sprachmodell und *sicher*, also nie ohne menschliche Freigabe und nie auf einer Live-Seite?

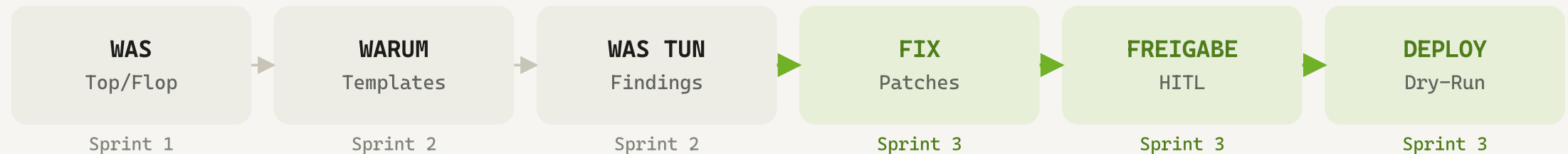


Abb. 2 – Sechs Akte in einem Lauf. **Sprint 3 ergänzt die letzten drei: FIX → FREIGABE → DEPLOY.**

Architektur: genau **ein neuer Port** (Publisher), ein neuer Agent — sonst unverändert.

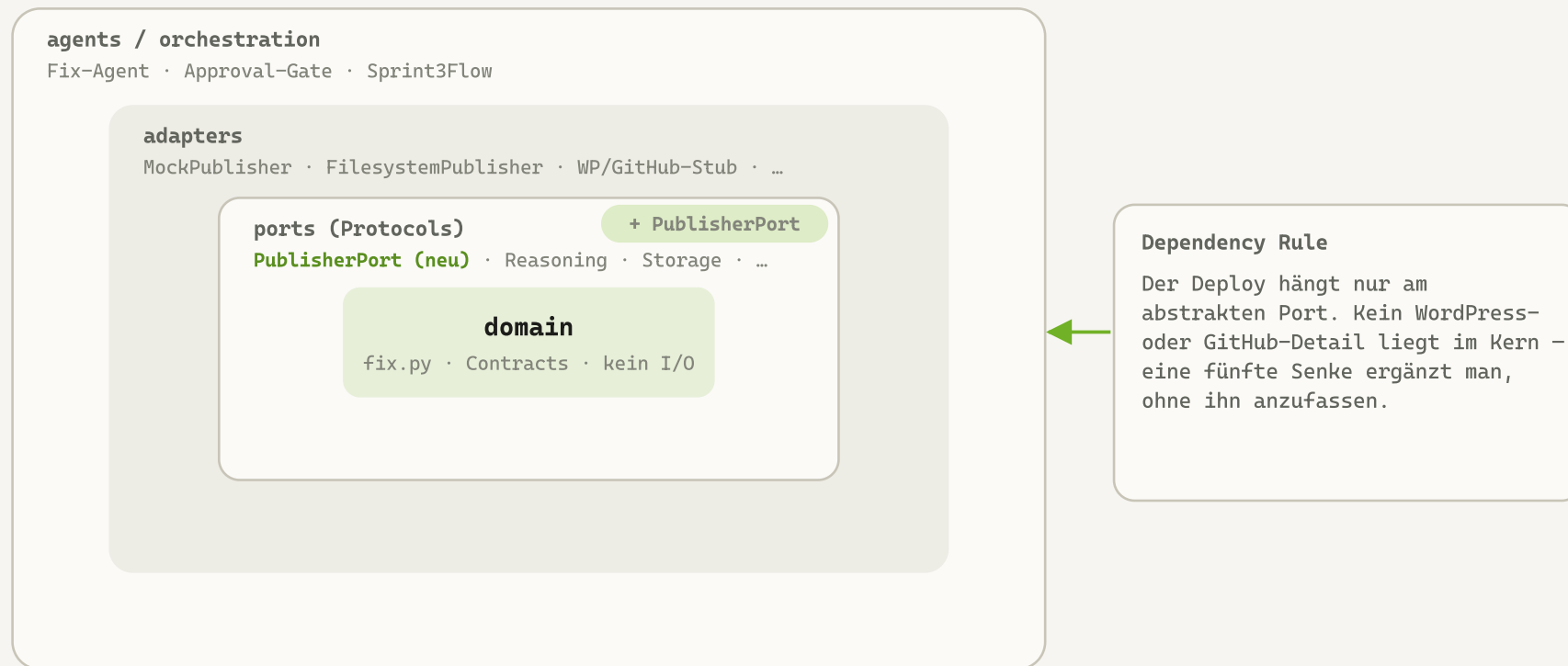


Abb. 3 – Hexagonale Schichten. **Neu: der PublisherPort** (+ Fix-Agent); die Dependency Rule bleibt unverändert.

Fix-Agent: aus jedem Finding ein **konkreter, belegter Patch**.

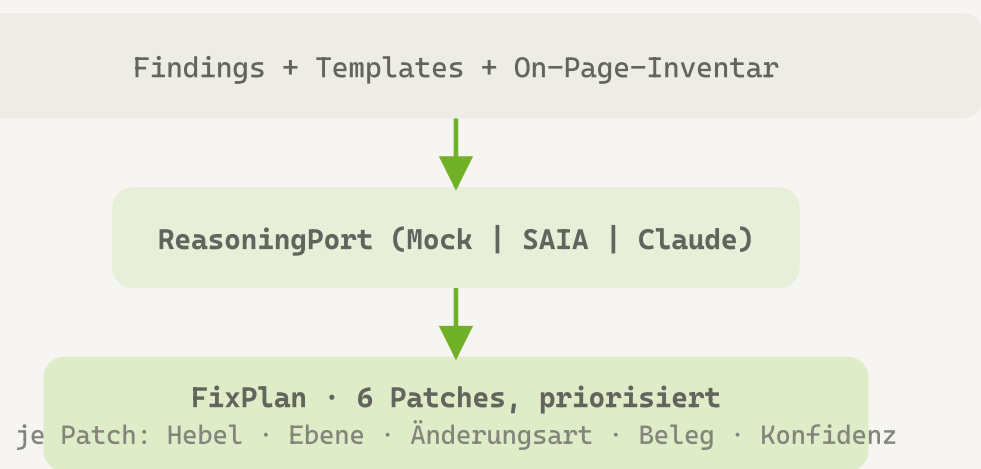


Abb. 4 – Spiegelt den GEO-Auditor: Retry, Per-Item-Validierung, Kosten-Tracking, Prompt `fix_agent.v1.md`.

Schema + /firewall-grundlagen

Definitionsblöcke · 0.74

FAQPage-JSON-LD: „Was ist eine Firewall?“ + 1-Satz-Antwort.

Beleg: keine maschinenlesbare Definition (Template t1).

Block + /password-manager-vergleich

Maschinenlesbar · 0.80

Vergleichstabelle (Open Source · 2FA · Preis) statt Fließtext.

Beleg: Vergleich nur als Prosa (Template t1).

- Ein Patch pro Finding — Herkunft (`finding_id`) + Template bleiben erhalten.
- `proposed_content` ist einsetzbar (Textblock / valides JSON-LD), nicht vage.
- Deterministische `patch_id` (`px-<finding>-<typ>`) → bit-genauer Fingerprint.

Human-in-the-Loop: kein Deploy ohne Freigabe — **in Code** **erzwungen.**



Abb. 5 – Das Gate liegt zwischen Plan und Deploy; aus einem `FixPlan` wird nie *direkt* ein `DeployResult` .

AUTOAPPROVEGATE

Gibt alle frei — die nicht-interaktive Demo (`--approve-all`).

INTERAKTIVER GATE

Fragt je Patch im Terminal; ohne TTY: sicherer Default = ablehnen.

REJECTALLGATE

Lehnt alles ab — Sicherheits-Demo & Test: 0 angewandt.

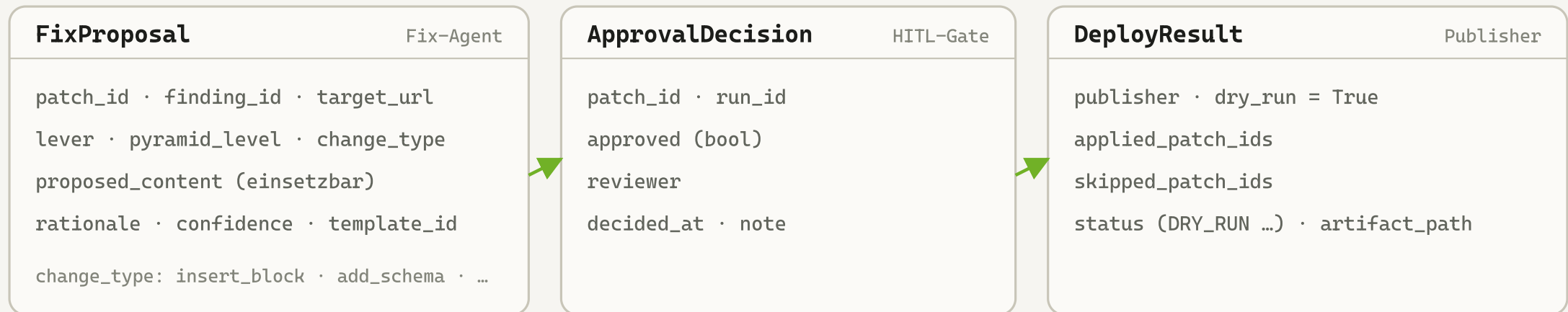
Projektregeln §6: Ein `Patch` wird nie ohne explizite menschliche Freigabe zu einem `DeployResult` — „auch nicht temporär zum Testen“. Doppelt abgesichert: Pipeline-Reihenfolge *und* Publisher-Vertrag.

Publisher-Port: ein Interface, drei Adapter — **sicher per Default.**



Abb. 6 – Harte Invariante (pro Adapter getestet): nicht freigegebene Patches landen **immer** in `skipped_patch_ids` ; `dry_run = true` in ganz Sprint 3.

Datenverträge: **geprüfte Strukturen** für Patch, Freigabe und Ergebnis.



Alle Modelle sind unveränderlich (frozen) · unbekannte Felder werden abgewiesen · patch_id deterministisch.

Abb. 7 – Die Übergaben zwischen Fix-Agent → Gate → Publisher sind Pydantic-Verträge, kein Freitext (Projektregeln §3.2).

Ergebnis eines **Live-Laufs** — it-sicherheit.de · Gemini + GPT-4o.

1 · WAS — GEMINI-CITATION-MESSUNG (GOOGLE-SEARCH-GROUNDING)

/phishing-erkennen		10×
/nis2-richtlinie		7×
/zero-trust-architektur		7×
/firewall-grundlagen		3×
/passwort-manager-vergleich		1×
/security-awareness-training		0×

2 · WARUM — GPT-4o-Muster: [t1] Integration von FAQ-/HowTo-Schemata (0.90).

3-5 · WAS TUN → FIX → FREIGABE → DEPLOY

Umschreiben /passwort-manager-vergleich px-f2-rewrite_block · 0.90

Faktendichte erhöhen: 2FA, Passwort-Generierung, Vergleich führender Manager (→ Template t2).

✓ **FREIGABE** cli:--approve-all Human-in-the-Loop

8 von 8 Patches freigegeben — kein Deploy ohne diese Entscheidung.

DEPLOY (Dry-Run)

Publisher **mock** · **DRY-RUN** · **KEINE EXTERNEN WRITES**

Patches **8 angewandt** · **0 übersprungen**

Live-Lauf: echte Gemini-Zitate + GPT-4o-Reasoning/Fix-Agent. Befehl: `uv run python -m geo_audit_loop --domain it-sicherheit.de --live --explain --fix --apply --approve-all` — Live-Läufe messen die reale Wirkung; den Determinismus-Beweis liefert der Offline-Lauf (Folie 11).

Demo-Ausgabe: der vollständige **Live-Lauf** mit einem Befehl.

```
$ uv run python -m geo_audit_loop --domain it-sicherheit.de --live --explain --fix --apply --approve-all
```

```
✓ Probe-Matrix sampeln (Gemini live) 48 Probes · 229.5 s  ✓ Fix-Agent (GPT-4o) 8 Patches · 22.6 s
✓ Crawl + Top/Flop                  8 Seiten                ✓ HITL-Freigabe      8 freigegeben
✓ Pattern-Miner (GPT-4o)            3 Templates · 7.4 s      ✓ Deploy             8 · Dry-Run
```

```
WAS      TOP 10x /phishing-erkennen · 7x /nis2-richtlinie · 7x /zero-trust-architektur
```

```
WARUM    [t1] Integration von FAQ-/HowTo-Schemata (Konfidenz 0.90)
```

```
FIX      [rewrite] /passwort-manager-vergleich · fact_density · 0.90
```

```
→ Faktendichte erhöhen: 2FA, Passwort-Generierung, Vergleich führender Manager
```

```
DEPLOY   Publisher mock · DRY-RUN · KEINE EXTERNEN WRITES · 8 angewandt · 0 übersprungen
```

```
Kosten/Lauf      48 Probes · 16713 Tokens · $0.1110
```

```
Report-Fingerprint b6ed70488487 (Live-Lauf – echte Modelle, nicht bit-reproduzierbar)
```

Live-Lauf: echte Gemini-Citation-Messung (Google-Search-Grounding) + GPT-4o-Reasoning. Die Kosten sind real (OpenAI). Live-Läufe variieren pro Lauf und sind nicht bit-reproduzierbar — der Determinismus-Beweis erfolgt offline auf der nächsten Folie.

Offline zweimal laufen → **derselbe Fingerprint** (inkl. Fix-Plan).

Der Live-Lauf (Folie 9–10) misst die *echte* Wirkung — variiert aber pro Lauf.

Den Beweis der Reproduzierbarkeit liefert der **Offline-Lauf** (Mock-Reasoning, Seed 42): der Fix-Plan fließt in den Hash ein, Freigabe/Deploy mit ihren Zeitstempeln bewusst nicht — bit-genau reproduzierbar.

6

Patches je Lauf
deterministische patch_id

100 %

freigegeben
(--approve-all)

0

externe Writes
(Dry-Run)

=

identischer Fingerprint
c1ecf54a2b27

Warum das funktioniert: Das Mock-Reasoning ist konstant, die `patch_id` wird abgeleitet (nicht zufällig), und `ApprovalDecision` / `DeployResult` bleiben mit ihren Zeitstempeln außerhalb des Hashes. Gleicher Seed ⇒ gleicher Fingerprint — als Integrationstest abgesichert.

Das Gate ist nicht nur Konvention — es ist **adversarial getestet**.

TEST 1 · OHNE FREIGABE

`apply_patches()` vor `await_approval()` → wirft **DeployBlocked**. Danach:
`load_deploy_result()` ist `None`, kein Artefakt geschrieben.

TEST 2 · REJECTALLGATE

Alle Patches abgelehnt → **0 angewandt**, alle in `skipped_patch_ids`, und der
`FilesystemPublisher` schreibt **null Dateien**.

```
# tests/integration/test_hitl_gate.py
def test_apply_without_approval_is_blocked(...):
    pipeline.propose_fixes() # Freigabe BEWUSST übersprungen
    with pytest.raises(DeployBlocked): pipeline.apply_patches()
    assert storage.load_deploy_result(...) is None
```

Kernaussage Sprint 3: Die wichtigste neue Anforderung — „kein Deploy ohne Freigabe“ — ist nicht dokumentiert, sondern in Code gegossen und durch einen harten Test bewiesen.

Definition of Done (DoD)

0

Fehler bei
mypy --strict (128 Dateien)

0

Befunde bei
ruff

182

grüne Tests
(+ 4 Live übersprungen)

3

Agenten mit
eigenem Eval-Slot

- Contracts, Publisher-Sicherheit, Fix-Agent, Storage, Fingerprint — je **Unit-getestet**.
- Der komplette **Sprint3Flow** + Reproduzierbarkeit als **Integrationstest**; das HITL-Gate adversarial.
- Bewusst **nicht** gebaut: der echte externe Write (WordPress/GitHub) — das ist Sprint 4.

Stand & Ausblick

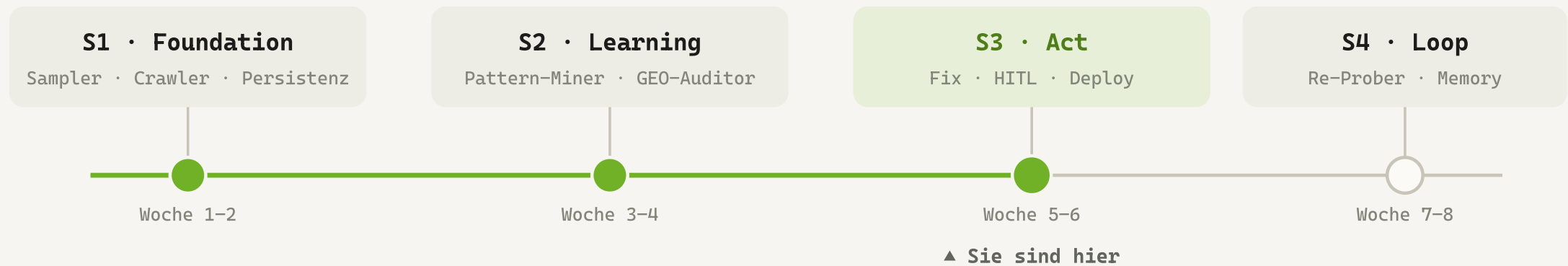


Abb. 8 – Drei von vier Sprints abgeschlossen. **Messen** ✓ · **Lernen** ✓ · **Handeln** ✓.

ERREICHT IN SPRINT 3

Fix · HITL · Dry-Run-Deploy

Aus `AuditFinding` wird ein freigegebener `Patch` und ein sicherer Deploy — bit-genau reproduzierbar, ohne je eine Live-Seite zu berühren.

SPRINT 4 · LOOP

Re-Probe + Memory

Echter WordPress-/GitHub-Deploy hinter dem bestehenden Port; nach 7/14/28 Tagen erneut messen → `EffectHypothesis` in Chroma — der Loop verfeinert künftige Fixes selbst.